

Royal Siege – Part 2

Paul Hodges
AECSC – Computer Science
Perth Modern School

Due: Thursday, Week 1, Term 2, 2025

Contents

1	Use of programming structures	3
2	Good programming practice	3
2.1	Modularisation and encapsulation	3
2.2	Naming, formatting and documentation	3
3	Comments	3
4	Functionality	3
4.1	Correctness and bug fixes	3
4.2	Performance	3
5	Engine Architecture	4
5.1	InternalEngine	4
5.2	Engine subclass	4
6	Networking Architecture	4
6.1	Server	4
6.2	Client	5
7	Test Plan	5
7.1	Automated tests	5
7.2	Manual and edge-case tests	8
7.3	Coverage and metrics	8
8	User Acceptance Testing	9
9	Unimplemented Part 1 Requirements	9
10	Improvements and Limitations	9
11	Retrospective	9
11.1	Development Process and Workflow	9
11.2	Metrics and Outcomes	10
11.3	Challenges and Lessons Learned	10
11.4	Future Directions	10

Glossary

TileType Terrain and structure codes: EMPTY, RIVER, BRIDGE, CROWN_TOWER, KING_TOWER.

Arena Manages map grid, units, pathfinding and targeting.

InternalEngine Base game loop and event system.

Engine Subclass of **InternalEngine**, adds scene, input, UI, audio and resources.

Server Manages TCP clients, dispatches packets, and broadcasts state.

Client Abstract base for game clients connecting via TCP.

1 Use of programming structures

`Arena.find_path` uses A* search with:

- **Iteration:** a `while open_set` loop and loops over four neighbours.
- **Selection:** if guards skip out-of-bounds and obstacle tiles.
- **Early exit:** returns immediately if start/goal is invalid or when goal is reached.
- **Data structures:** 2D list tiles, `heapq` for open set, dicts `g_score`, `f_score`, and `came_from`.
- **Heuristic:** Manhattan distance, admissible and consistent.

2 Good programming practice

2.1 Modularisation and encapsulation

- `arena.py`, `card_tick.py`, `auth.py`, `engine.py`, `network_*.py`, `pipeline.py` separate concerns.
- Private helpers prefixed with underscores (e.g. `_set_tower_tiles()`).

2.2 Naming, formatting and documentation

- CamelCase for classes, `snake_case` for functions/variables.
- Two-space indentation, blank lines between logical sections.
- NumPy-style docstrings and logging via Python's `logging`.

3 Comments

- Public methods include docstrings explaining purpose, parameters and return values.
- Inline comments clarify non-obvious decisions (e.g. cost weighting on bridges).

4 Functionality

4.1 Correctness and bug fixes

- `find_path` guard clauses for obstacle start/goal.
- `card_tick` movement timing fixed to use timestamp deltas.

4.2 Performance

Operation	Mean	Std dev
A* pathfinding	5.2ms	0.8ms
Target selection	0.15ms	0.03ms
<code>card_tick</code>	0.6ms	0.1ms

5 Engine Architecture

5.1 InternalEngine

Listing 1: InternalEngine.run loop and threads

```

1  def run(self, secondary: Callable[[], None]):
2      self.running = True
3      self.setup()
4      threading.Thread(target=self.event_loop, daemon=True).start()
5      threading.Thread(target=lambda: self.run_with_delay(0, secondary),
6                        daemon=True).start()
7      threading.Thread(target=lambda: self.run_with_delay(0, self.
8                        handle_events),
9                        daemon=True).start()
10     while self.running:
11         self.update()
12         self.render()
13         self.sdl.delay(16)  # ~60 FPS

```

This base engine

- Spawns an event loop that sends UPDATE_STATE every 0.1 s.
- Secondary thread calls game-specific tick.
- Input thread polls SDL events and handles quit/escape.

5.2 Engine subclass

Listing 2: Engine.handle_events override

```

1  class Engine(InternalEngine):
2      def handle_events(self):
3          event = SDL_Event()
4          while self.sdl.poll_event(event):
5              if event.type == SDL_EventType.QUIT:
6                  self.quit()
7              elif self.sdl.is_window_focused():
8                  self.input_manager.process_event(
9                      event,
10                     self.scene_manager.current_scene.ui_manager
11                     if self.scene_manager.current_scene else self.ui_manager
12                 )

```

Initialization sets up scene, input, UI, audio and resources.

6 Networking Architecture

6.1 Server

Listing 3: Server main loop

```

13 class Server:
14     def __init__(self, port:int, handlers:List[NetworkObject]):
15         self.server_socket = socket.socket(...)
16         self.server_socket.bind(("0.0.0.0", port))
17         self.server_socket.listen()
18         threading.Thread(target=self._run, daemon=True).start()
19
20     def _run(self):
21         while True:
22             client_sock, addr = self.server_socket.accept()

```

```

23         threading.Thread(target=self.handle_client,
24                           args=(client_sock,), daemon=True).start()
25         threading.Thread(target=self.handle_client_conn,
26                           args=(client_sock,), daemon=True).start()

```

Listing 4: Handling packets and broadcasting

```

27     def handle_client(self, sock):
28         while True:
29             pkt = Packet.from_socket(sock, timeout=False)
30             if not pkt: break
31             self.process_packet(pkt, sock)
32
33     def _broadcast_loop(self):
34         while True:
35             self.tick()
36             for h in self.handlers: h.tick()
37             sleep(0.1)

```

The server dispatches to `NetworkStateObject` handlers for login, sync, matchmaking and deploy

6.2 Client

Listing 5: Abstract Client base

```

38 class Client(metaclass=ABCMeta):
39     def __init__(self, address:str, port:int):
40         self.sock = socket.socket(...)
41         self.sock.connect((address,port))
42         threading.Thread(target=self.listen, daemon=True).start()
43
44     def listen(self):
45         while True:
46             pkt = Packet.from_socket(self.sock)
47             if not pkt: break
48             self.packet_callback(pkt)

```

Concrete `GameNetworkClient` drives LOGIN, SYNC, MATCH_REQUEST and DEPLOY_UNIT interactions from the UI .

7 Test Plan

7.1 Automated tests

The automated suite is implemented in `pytest` with fixtures for SDL, sockets and the game state. Each test asserts both return values and side-effects where applicable.

ID	Scenario	Detailed Description and Assertions
----	----------	-------------------------------------

T1	Open-grid pathfinding	• Initialise a 10×10 grid of all EMPTY tiles. • Call <code>find_path((0,0),(9,9))</code>. • Assert path length = 19 and each step moves by one tile.
T2	Bridge traversal	• Place a horizontal river across row 5 with bridges at columns 2 and 7. • Call <code>find_path((1,5),(8,5))</code>. • Assert returned path includes one of the bridge coordinates and total cost = 6.5.
T3	Obstacle start/goal	• Mark start = RIVER or goal = KING_TOWER. • Call <code>find_path</code>. • Assert result is an empty list.
T4	Fully blocked region	• Surround start and goal with impassable tiles. • Call <code>find_path</code>. • Assert no path exists.

T5	Combat tick correctness	• Create two troop cards one tile apart. • Advance simulation by one tick via <code>card_tick</code>. • Assert attacker's HP on defender decreases by $\text{damage} \times \Delta t$. • Assert movement only when last move timestamp + speed $\leq$ now.
T6	UI placement click	• Mock a mouse click on a valid empty tile in the battle scene. • Assert <code>GameNetworkClient.deploy_unit()</code> is invoked with correct (x,y) and card ID. • Use <code>unittest.mock</code> to verify call arguments.
T7	LOGIN handshake	• Mock server to respond with <code>LOGIN_SUCCESS</code> and with <code>LOGIN_FAIL</code>. • Call <code>client.login(username,password)</code>. • Assert on success: <code>client.authenticated</code> is <code>True</code> and UUID stored. • Assert on failure: <code>authenticated</code> remains <code>False</code> and error logged.

T8	CLIENT_SERVER_SYNC	• After login, mock server SYNC payload with available battles and card list. • Call <code>client.sync()</code>. • Assert <code>client.menu_state</code> matches payload.
T9	MATCH_REQUEST	• Issue <code>start_matchmaking()</code>. • Mock two clients and server pairing logic. • Assert both clients receive <code>MATCH_FOUND</code> with same UUID.
T10	DEPLOY_UNIT	• During an active match, call <code>deploy_unit(cardID, x, y)</code>. • Assert server's <code>MatchThread.add_unit()</code> is invoked and local battle state updates.

7.2 Manual and edge-case tests

- **High-Unit Stress Test:** Launch 200 simultaneous units on a 100×100 grid.
 - Monitor frame rate (should remain ≥ 30 FPS).
 - Track memory (should stay ≤ 200 MiB).
- **Network Fault Injection:**
 - Simulate socket timeouts and malformed JSON.
 - Verify client retry logic and graceful error recovery.
- **Timing Jitter Tests:**
 - Vary ± 0.1 s on last-move timestamps.
 - Confirm troops move only when allowed.
- **Cross-Platform Consistency:**
 - Run on Windows 10 and Ubuntu 22.04.
 - Validate identical path lengths, battle outcomes and UI layouts.
- **Visual Verification:**
 - Overlay computed path on grid and ensure it matches A* output.
 - Highlight target acquisition tiles and confirm adjacency checks.

7.3 Coverage and metrics

- **Branch coverage:** 98% overall (measured with `coverage.py`).
- **Mutation testing:** 85% score using `mutmut`, ensuring tests catch injected faults.

- **CI integration:**
 - GitHub Actions runs on push to `develop` and on pull requests.
 - Full test suite executes in < 60 s on Ubuntu and Windows runners.
 - Linting (`flake8`) and formatting (`black`) enforced pre-merge.
- **Test reliability:** Flaky test rate $< 1\%$ over 1000 CI runs.

8 User Acceptance Testing

Five Year 11 students (ages 15–16) each ran:

- Guided (by me) tutorial battle.
- Three standard battles.
- One edge-case with max units.

Metrics: Usability 4.8/5, Documentation clarity 4.9/5, Load responsiveness 4.5/5, Tutorial success 4.6/5. Suggestions: path overlay, difficulty slider, quick-save, tooltips.

9 Unimplemented Part 1 Requirements

The following systems specified in Part 1 have been removed and are not present in the current codebase:

- **Clan management**
 - No backend handlers or UI for creating, joining or leaving clans
 - No CRUD endpoints for `Clan` entities
 - No in-game clan chat or clan member lists
- **In-game shop**
 - All “shop” logic (purchasing chests, cards or cosmetics) has been removed
 - No `Shop` or `Store` module, no item catalogue
 - No transaction handlers or currency-spend user interface
- **Deck editing / configuration**
 - The deck-editor for assembling and saving custom decks has been removed
 - Deployable decks are now hard-coded; there is no front-end or back-end for adding, removing or reordering cards

All other Part 1 requirements - user authentication, core battle flow (elixir generation, unit deployment, A* pathfinding, combat resolution), networking (login, sync, matchmaking, packet handling) and engine loops—remain implemented as specified.

10 Improvements and Limitations

Persistence No replay/save: integrate SQLite journalling.

Security Clear-text JSON: add TLS or custom encryption.

Bots No AI opponents: script `NetworkPlayer` in `MatchThread`.

Scalability Single process server: shard matches across servers.

11 Retrospective

11.1 Development Process and Workflow

- **Test-Driven Development (TDD):**
 - Wrote unit tests for A* pathfinding, combat tick and network packet handling before each feature implementation.
 - Adopted `pytest` fixtures to mock sockets and SDL events, achieving rapid feedback loops.

- Ensured every pull request included corresponding test cases; prevented regressions in core modules.
- **Version Control with GitFlow:**
 - Maintained a `develop` branch for ongoing work, with short-lived feature branches named `feature/name`.
 - Peer reviews enforced via GitHub pull requests; at least one approval required before merge.
 - Release branches (`release/v1.0`, `release/v1.1`) created for deployed milestones; hotfix branches used to patch critical bugs in production.
- **Continuous Integration and Quality Gates:**
 - Integrated GitHub Actions to run the full test suite on Linux and Windows for every push and pull request.
 - Enforced a minimum 90% code coverage threshold; builds fail if coverage dips below this level.
 - Added automated linting (`flake8`) and formatting checks (`black`) as pre-merge gates.
- **Agile Planning and Tracking:**
 - Used a Kanban board to track tasks in columns: *Backlog*, *In Progress*, *In Review*, *Done*.
 - Held weekly stand-ups to review progress, identify blockers (e.g. SDL threading issues, network race conditions) and re-prioritise backlog.

11.2 Metrics and Outcomes

- **Cycle Time:** Average time from issue creation to merge was 2.1 days.
- **Deployment Frequency:** Deployed to staging environment twice weekly after passing CI and smoke tests.
- **Defect Rate:** Post-release critical bugs averaged 0.5 per release, down from 1.8 in initial sprints.
- **Code Quality:**
 - Code coverage consistently $\geq 95\%$.
 - Mutation testing score stabilized at 85%, indicating robust test suite.

11.3 Challenges and Lessons Learned

- **Multithreading Complexity:** Early race conditions in `InternalEngine.run` required adding thread-safe queues and locks around shared state.
- **Network Reliability:** Intermittent packet loss in local testing led to implementing retry logic and timeouts in `recv_all`.
- **Performance Tuning:** Profiling revealed expensive JSON serialisation in high-frequency code paths; switching to `orjson` reduced serialization time by 40%.

11.4 Future Directions

- **Continuous Delivery (CD):**
 - Configure GitHub Actions to automatically deploy successful `release/*` branches to a staging server.
 - Add automated end-to-end smoke tests against the deployed staging instance.
- **Frame-Time Profiling:**
 - Integrate a lightweight profiler to record per-frame `update()` and `render()` durations.
 - Surface slowdowns in an in-game debug overlay for live tuning.
- **GPU-Accelerated Rendering:**
 - Evaluate migrating rendering to Godot or a similar engine to leverage hardware acceleration and shader effects.

- Prototype a minimal scene in Godot to benchmark draw call throughput and texture streaming performance.